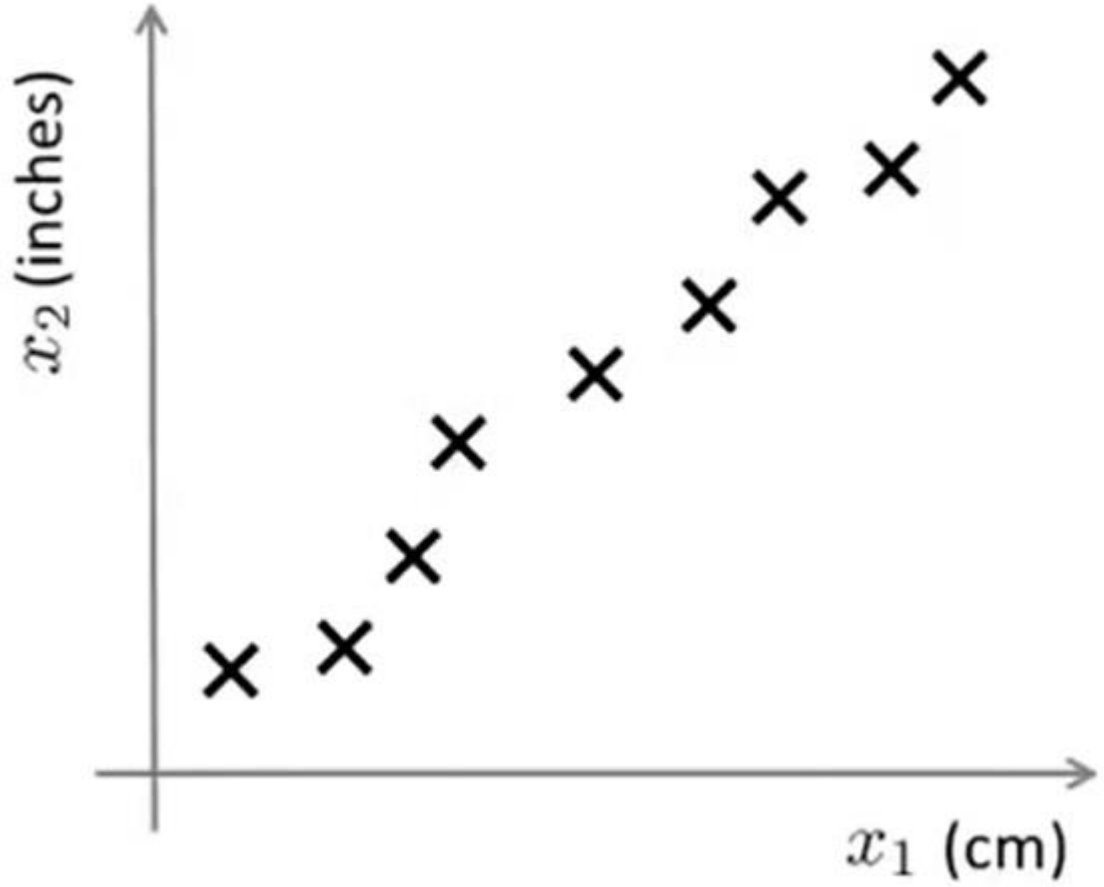
Lecture 19: Dimensionality Reduction

Md. Shahriar Hussain
ECE Department, NSU

Dimensionality Reduction

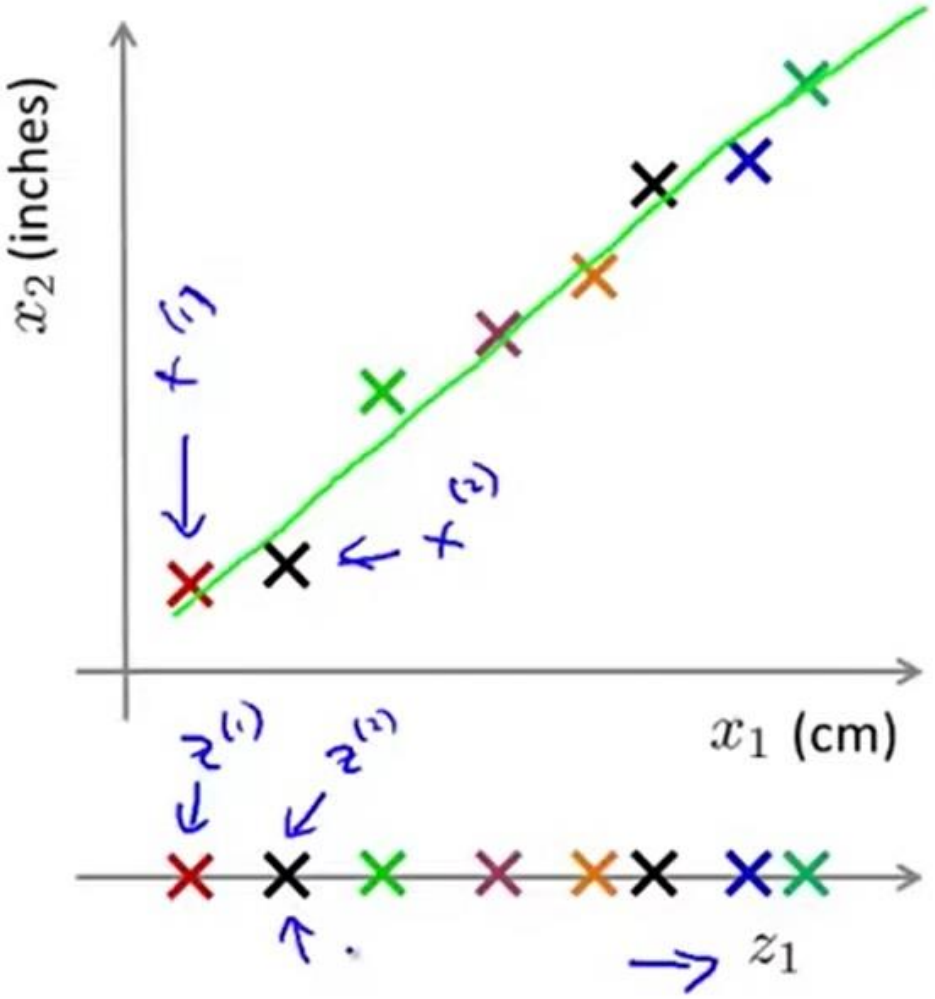
- Many Machine Learning problems involve thousands or even millions of features for each training instance.
- Not only do all these features make training extremely slow, but they can also make it much harder to find a good solution
- This problem is often referred to as the ***curse of dimensionality***
- It is often possible to reduce the number of features considerably
- Apart from **speeding up** training, dimensionality reduction is also extremely useful for **data visualization**

Dimensionality Reduction



Reduce data from 2D to 1D

Dimensionality Reduction



Reduce data from 2D to 1D

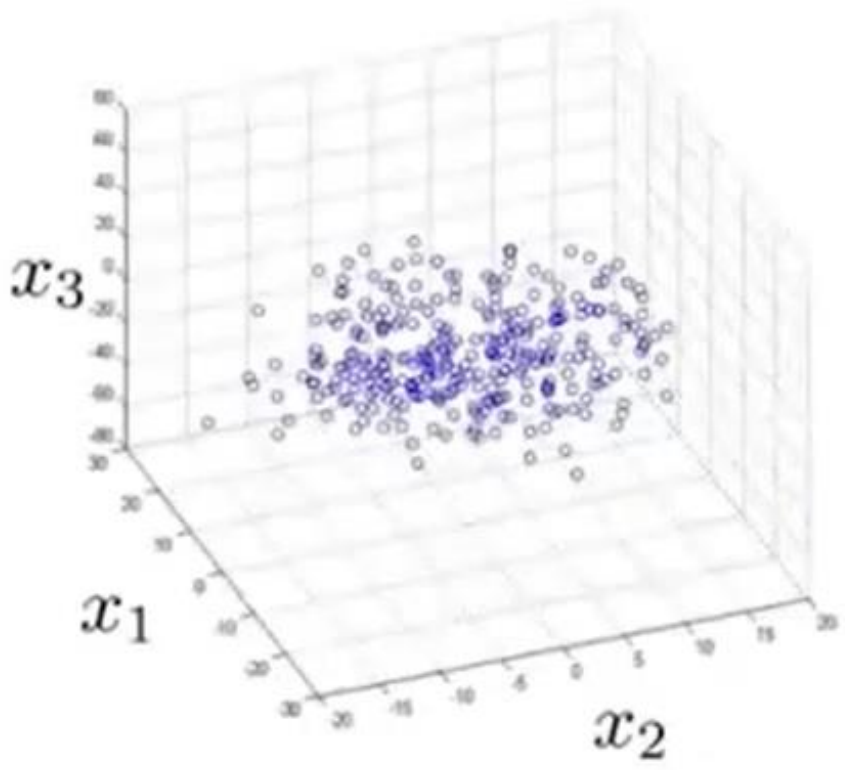
$x^{(1)} \in \mathbb{R}^2 \rightarrow z^{(1)} \in \mathbb{R}$

$x^{(2)} \in \mathbb{R}^2 \rightarrow z^{(2)} \in \mathbb{R}$

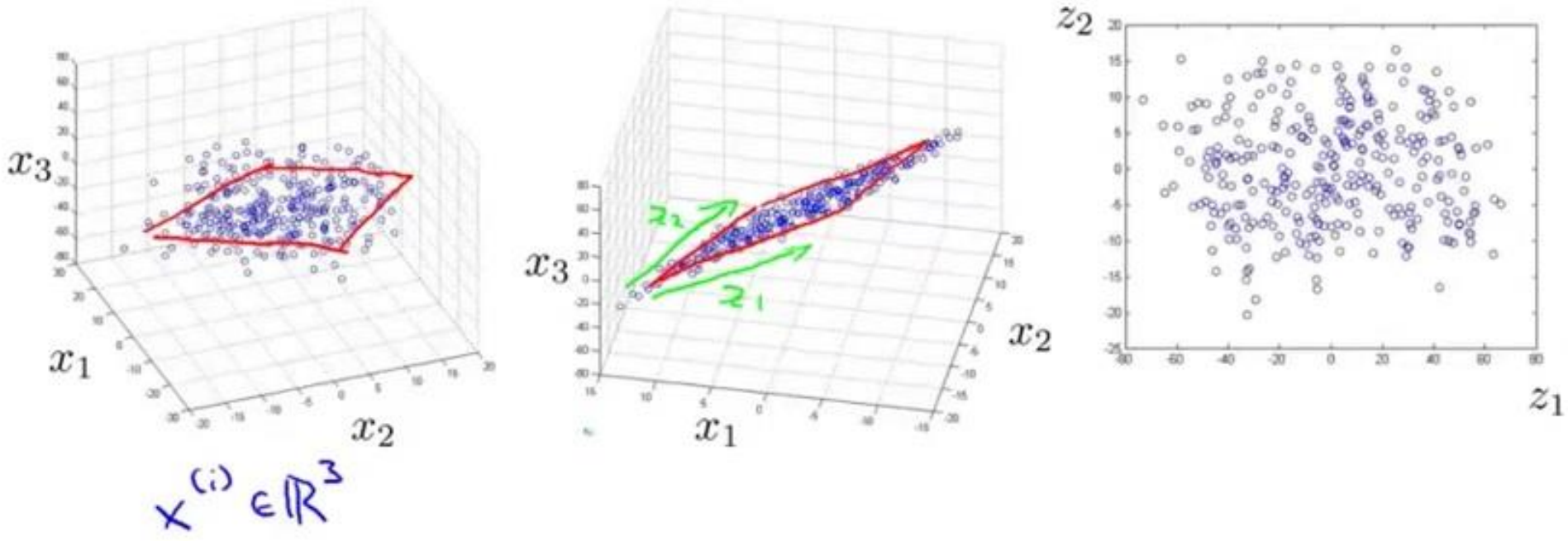
$\vdots$

$x^{(m)} \rightarrow z^{(m)}$

Dimensionality Reduction 3D to 2D



Dimensionality Reduction 3D to 2D



Projection for Dimensionality Reduction

- In most real-world problems, training instances are *not spread out uniformly across* all dimensions. Many features are almost constant, while others are highly correlated

- As a result, all training instances lie within (or close to) a much lower-dimensional *subspace of the high-dimensional space.*

- If we **project every training instance** perpendicularly onto this subspace, we can reduce the dimension

Principal Component Analysis (PCA)

- **Principal Component Analysis (PCA)** is by far the most popular dimensionality reduction algorithm.

- First **it identifies the hyperplane** that lies closest to the data, and then it projects the data onto it

- PCA identifies **the axis** that accounts for the **largest amount of variance** in the training set

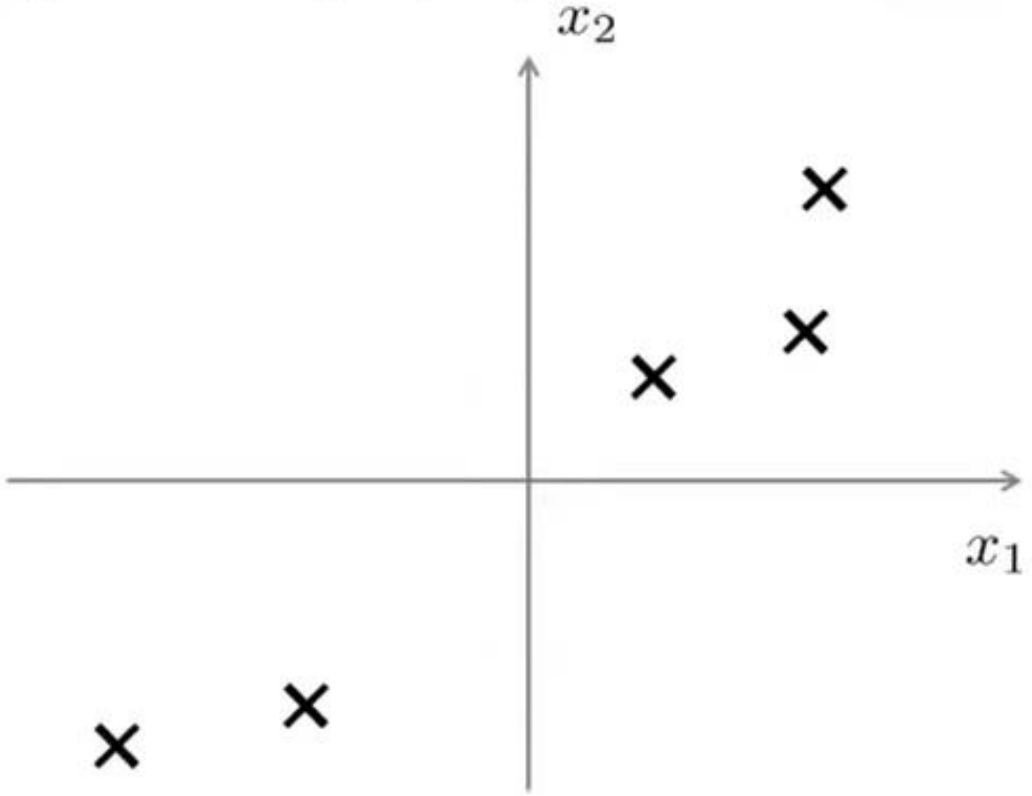
Principal Component Analysis (PCA)

- It seems reasonable to select the axis that preserves **the maximum amount of variance**, as it will most likely **lose less information** than the other projections

- Another way to justify this choice is that **it is the axis that minimizes the mean squared distance between the original dataset and its projection** onto that axis. This is the rather simple idea behind PCA

PCA

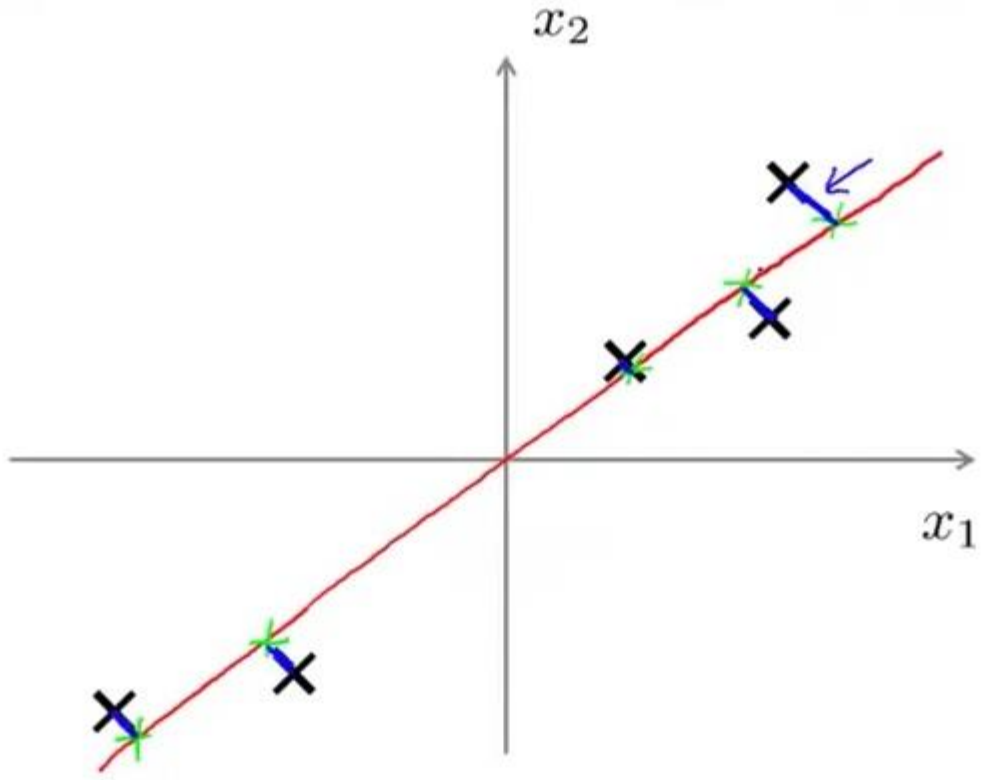
Principal Component Analysis (PCA) problem formulation



$x \in \mathbb{R}^2$

PCA

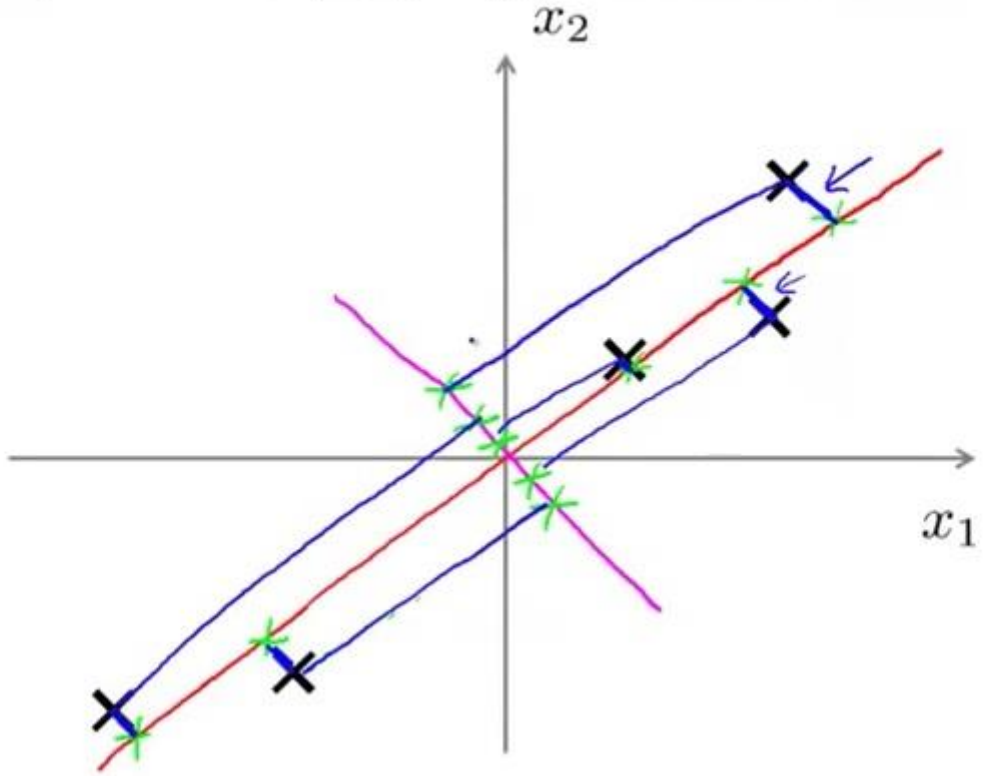
Principal Component Analysis (PCA) problem formulation



$x \in \mathbb{R}^2$

PCA

Principal Component Analysis (PCA) problem formulation

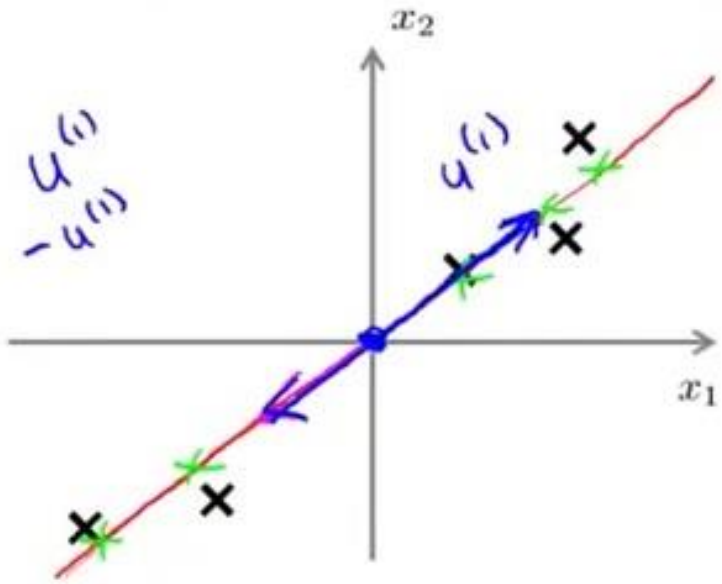


$x \in \mathbb{R}^2$

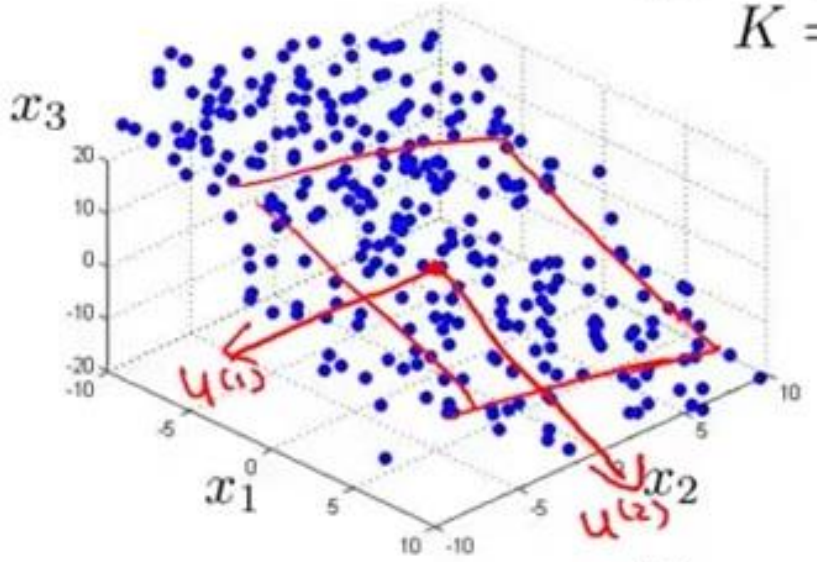
Choose the axis that minimizes the mean squared distance between the original dataset and its projection onto that axis

PCA

Principal Component Analysis (PCA) problem formulation



$3D \rightarrow 2D$
$K = 2$

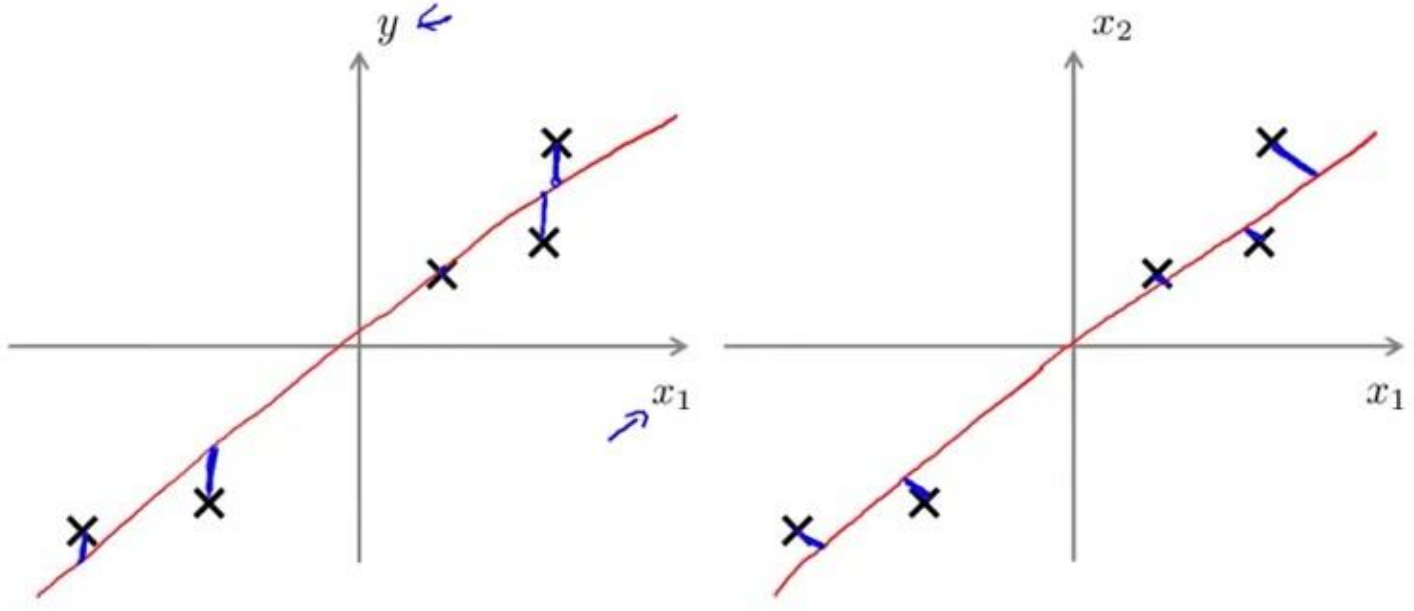


Reduce from 2-dimension to 1-dimension: Find a direction (a vector $u^{(1)} \in \mathbb{R}^n$) onto which to project the data so as to minimize the projection error.

Reduce from n-dimension to k-dimension: Find k vectors $u^{(1)}, u^{(2)}, \ldots, u^{(k)}$ onto which to project the data, so as to minimize the projection error.

PCA vs Linear Regression

PCA is not linear regression



- In linear regression, we find a hypothesis where mean square distance will be minimum
- We try to predict the target value y

- In PCA we try to find out an axis where the projection and actual datas distance will be minimum.
- We are not predicting anything. Rather reducing dimensionality

PCA Data Preprocessing

Data preprocessing

Training set: $x^{(1)}, x^{(2)}, \ldots, x^{(m)}$

Preprocessing (feature scaling/mean normalization):

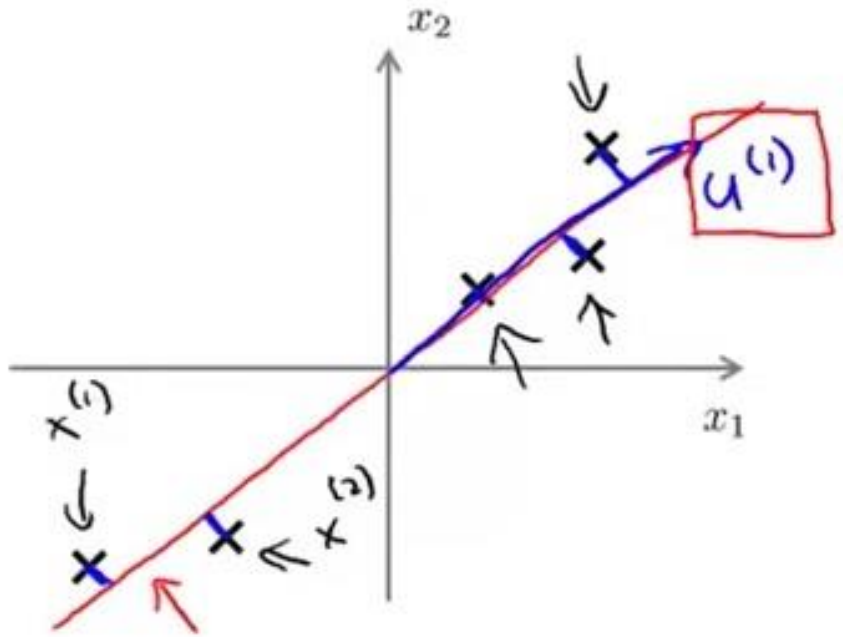
$$\mu_j = \frac{1}{m} \sum_{i=1}^{m} x_j^{(i)}$$

Replace each $x_j^{(i)}$ with $x_j - \mu_j$.

If different features on different scales (e.g., $x_1 =$ size of house, $x_2 =$ number of bedrooms), scale features to have comparable range of values.

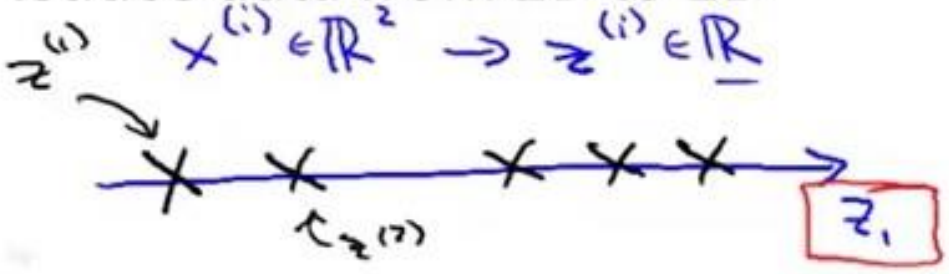
$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \mu_j}{s_j}$$

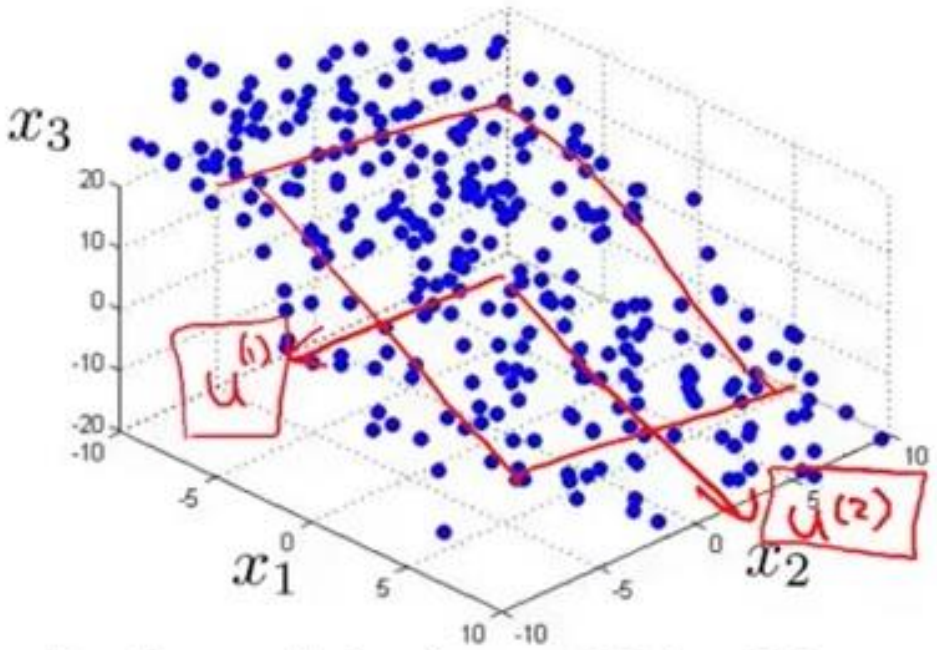
PCA Algorithm



Reduce data from 2D to 1D

$x^{(i)} \in \mathbb{R}^2 \rightarrow z^{(i)} \in \mathbb{R}$





Reduce data from 3D to 2D

$x^{(i)} \in \mathbb{R}^3 \rightarrow z^{(i)} \in \mathbb{R}^2$

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$$

PCA Algorithm

Reduce data from n-dimensions to k-dimensions

Compute "covariance matrix":

Sigma $\Sigma = \frac{1}{m} \sum_{i=1}^{n} (x^{(i)})(x^{(i)})^T$

Compute "eigenvectors" of matrix Σ

- The Eigenvector which corresponds to the maximum Eigenvalue of the Covariance matrix, will be the direction along which the dataset has the most information

- The largest eigenvector of of the Covariance matrix corresponds to the principal component of the data

PCA Algorithm

Reduce data from n-dimensions to k-dimensions

Compute "covariance matrix":

$$\Sigma = \frac{1}{m}\sum_{i=1}^{n}(x^{(i)})(x^{(i)})^T$$

$(x^{(i)})$: $n \times 1$, $(x^{(i)})^T$: $1 \times n$, result: $n \times n$

Compute "eigenvectors" of matrix Σ

$$U = \begin{bmatrix} | & | & | & & | \\ u^{(1)} & u^{(2)} & u^{(3)} & \cdots & u^{(m)} \\ | & | & | & & | \end{bmatrix}$$

k

$U \in \mathbb{R}^{n \times n}$

$u^{(1)}, \ldots, u^{(k)}$

PCA Algorithm

$$U = \underbrace{\begin{bmatrix} | & | & & | \\ u^{(1)} & u^{(2)} & \cdots & u^{(n)} \\ | & | & & | \end{bmatrix}}_{k} \in \mathbb{R}^{n \times n}$$

$x \in \mathbb{R}^n \longrightarrow z \in \mathbb{R}^k$

$$z^{(i)} = \underbrace{\begin{bmatrix} | & | & & | \\ u^{(1)} & u^{(2)} & \cdots & u^{(k)} \\ | & | & & | \end{bmatrix}^T}_{n \times k} \quad x^{(i)} = \underbrace{\begin{bmatrix} — (u^{(1)})^T — \\ \vdots \\ — (u^{(k)})^T — \end{bmatrix}}_{k \times n} \underbrace{x^{(i)}}_{n \times 1}$$

$z \in \mathbb{R}^k$

U_{reduce}

$k \times 1$

Scikit-Learn for PCA

- The following code applies PCA to reduce the dimensionality of the dataset down to two dimensions

```
from sklearn.decomposition import PCA
pca = PCA(n_components = 2)
X2D = pca.fit_transform(X)
```

Choosing the Right Number of Dimensions

- Instead of arbitrarily choosing the number of dimensions to reduce down to, it is simpler to choose the number of dimensions that add up to a sufficiently large portion of the variance (e.g., 95%)

- Unless, of course, we are reducing dimensionality for data visualization—in that case we will want to reduce the dimensionality down to 2 or 3.

Choosing the Right Number of Dimensions

- The following code performs PCA without reducing dimensionality, then computes the minimum number of dimensions required to preserve 95% of the training set's variance

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```